IMAGE CAPTIONING USING A PRE-TRAINED MACHINE LEARNING MODEL

Task 3 - Machine Learning Model Exploration

Internship Project Report

Organization:
InAmigos Foundation

Submitted by:
Debayan Manna

2. PROJECT OVERVIEW

This project implements an Image Captioning system. The system takes an image file as input—either via a terminal command line interface or uploaded through a local web server—and automatically generates a natural-language English caption describing the visual content of that image.

3. OBJECTIVE

The primary objective of this task is to explore how a machine learning model can understand visual image content and translate it into textual descriptions. By leveraging transfer learning, this project demonstrates how to perform inference on complex data without requiring the massive compute resources necessary to train a model from scratch.

4. TECHNOLOGIES USED

Technology	Purpose
Python	Core programming language used for the application logic and web server.
PyTorch	Deep learning framework required for underlying tensor computations.

Technology	Purpose
Hugging Face Transformers	Library used to load the pre-trained machine learning model.
BLIP	The specific vision-language model (`Salesforce/blip-image-captioning-base`) used.
Pillow (PIL)	Library used for opening and processing the input images.

5. HOW IT WORKS

The application workflow follows a clean inference pipeline:

```
        [Input Image]
              ↓
    [Image Preprocessing]
              ↓
[Pre-trained Image Captioning Model]
              ↓
     [Caption Generation]
              ↓
      [Generated Caption]
```

This project uses **BLIP (Bootstrapping Language-Image Pre-training)**. BLIP is a state-of-the-art vision-language model. In this project, the BLIP model was not trained from scratch; rather, the pre-trained model weights were downloaded and used exclusively for **model inference** to generate text based on visual features.

---

6. IMPLEMENTATION

The implementation is structured into a simple, functional pipeline:

1. **Load the pre-trained model:** The script initializes the `BlipProcessor` and `BlipForConditionalGeneration` model.
2. **Load the input image:** The user provides an image, which is opened using the Pillow library and converted to RGB.
3. **Process the image:** The processor converts the raw image into mathematical tensors.
4. **Generate the caption:** The model analyzes the tensors and predicts a sequence of text tokens, which are decoded into a readable string.
5. **Display/save the generated caption:** The final string is printed to the user and saved to a local `captions.txt` log file.

---

7. TESTING AND RESULTS

The model was tested using diverse sample images.

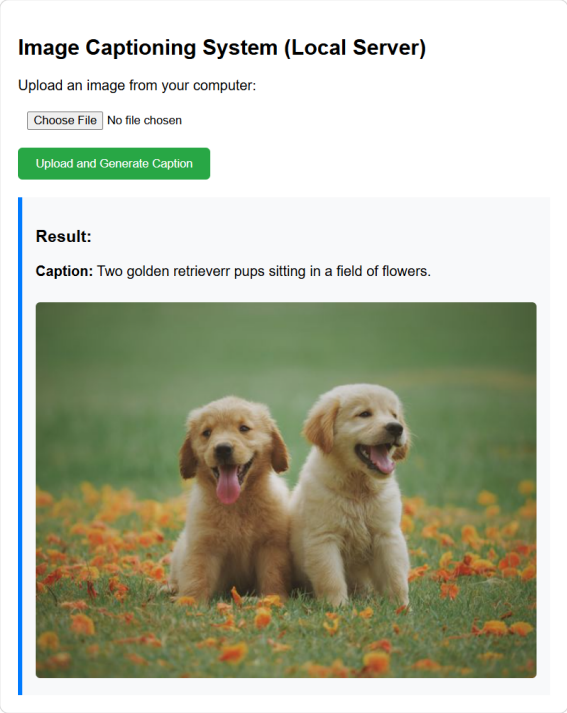
Test Image	Generated Caption	Observation
`image1.jpg` (Person)	"A woman with long hair and a green and white patterned shirt."	The model successfully identified the main subject and specific clothing patterns.
`image2.jpg` (Animals)	"Two golden retriever pups sitting in a field of flowers."	The model correctly identified the species, quantity, and background environment.
`image3.jpg` (Landscape)	"A small cabin sits on the edge of a lake."	The model accurately described inanimate objects and the scenery setting.

The pre-trained model performed exceptionally well during testing, accurately capturing the main subjects and the context of the scenes without requiring any local training.

---

8. SCREENSHOTS

System Execution and Generated Result:



(Caption: The system running on a local web server, successfully generating a descriptive caption for the uploaded sample image.)

---

9. LEARNING OUTCOMES

Through this internship task, the following concepts were explored and applied:

- Image captioning logic and architecture.
- Integration of computer vision with natural language generation.
- Utilizing pre-trained machine learning models via Hugging Face.
- Performing model inference on local hardware.

- Image preprocessing using Python tools.
- Testing and evaluating ML predictions on unseen data.

---

10. LIMITATIONS

- **Accuracy Constraints:** Generated captions may occasionally be inaccurate or miss highly specific background details.
- **Complex Scenes:** Highly cluttered or unusual scenes may not always be described correctly by the model.
- **Hardware Dependency:** Model performance and inference speed depend heavily on the available system hardware.

---

11. CONCLUSION

This project successfully demonstrates the implementation of a functional Image Captioning system. By utilizing a pre-trained BLIP model for inference, the project efficiently bridges computer vision and natural language processing. The task provided valuable hands-on experience in loading complex AI models, processing real-world image data, and handling prediction pipelines, forming a strong foundation in modern machine learning application development.

---

12. REFERENCES

- Hugging Face Transformers: https://huggingface.co/docs/transformers/
- BLIP Model Card (Salesforce): https://huggingface.co/Salesforce/blip-image-captioning-base
- PyTorch: https://pytorch.org/
- Pillow (Python Imaging Library): https://pillow.readthedocs.io/